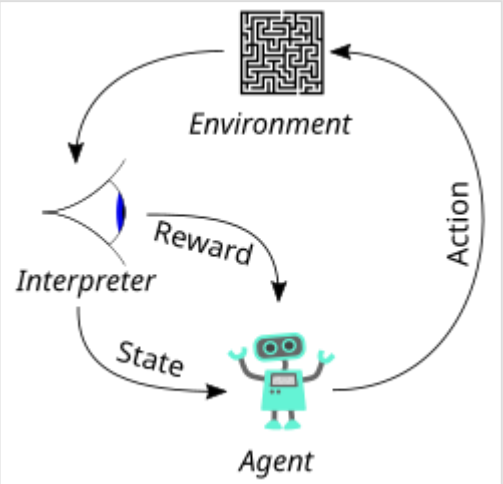
# Reinforcement learning

In machine learning and optimal control, **reinforcement learning** (**RL**) (*also known as approximate dynamic programming*)[1] is concerned with how an intelligent agent should take actions in a dynamic environment in order to maximize a reward signal. Reinforcement learning is one of the three basic machine learning paradigms, alongside supervised learning and unsupervised learning.




The typical framing of a reinforcement learning (RL) scenario: an agent takes actions in an environment, which is interpreted into a reward and a state representation, which are fed back to the agent.

While supervised learning and unsupervised learning algorithms respectively attempt to discover patterns in labeled and unlabeled data, reinforcement learning involves training an agent through interactions with its environment. To learn to maximize rewards from these interactions, the agent makes decisions between trying new actions to learn more about the environment (exploration), or using current knowledge of the environment to take the best action (exploitation).[2] The search for the optimal balance between these two strategies is known as the exploration–exploitation dilemma.

The environment is typically stated in the form of a Markov decision process, as many reinforcement learning algorithms use dynamic programming techniques.[3] The main difference between classical dynamic programming methods and reinforcement learning algorithms is that the latter do not assume knowledge of an exact mathematical model of the Markov decision process, and they target large Markov decision processes where exact methods become infeasible.[4]

## Principles

Due to its generality, reinforcement learning is studied in many disciplines, such as game theory, control theory, operations research, information theory, simulation-based optimization, multi-agent systems, swarm intelligence, and statistics. In the operations research and control literature, RL is called *approximate dynamic programming*, or *neuro-dynamic programming*. The problems of interest in RL have also been studied in the theory of optimal control, which is concerned mostly with the existence and characterization of optimal solutions, and algorithms for their exact computation, and less with learning or approximation (particularly in the absence of a mathematical model of the environment).

Basic reinforcement learning is modeled as a Markov decision process:

- A set of environment and agent states (the state space), $\mathcal{S}$;
- A set of actions (the action space), $\mathcal{A}$, of the agent;

- $P_a(s, s') = \Pr(S_{t+1}=s' \mid S_t=s, A_t=a)$, the transition probability (at time $t$) from state $s$ to state $s'$ under action $a$.
- $R_a(s, s')$, the immediate reward after transitioning from $s$ to $s'$ under action $a$.

The purpose of reinforcement learning is for the agent to learn an optimal (or near-optimal) policy that maximizes the reward function or other user-provided reinforcement signal that accumulates from immediate rewards. This is similar to processes that appear to occur in animal psychology. For example, biological brains are hardwired to interpret signals such as pain and hunger as negative reinforcements, and interpret pleasure and food intake as positive reinforcements. In some circumstances, animals learn to adopt behaviors that optimize these rewards. This suggests that animals are capable of reinforcement learning.[5][6]

A basic reinforcement learning agent interacts with its environment in discrete time steps. At each time step $t$, the agent receives the current state $S_t$ and reward $R_t$. It then chooses an action $A_t$ from the set of available actions, which is subsequently sent to the environment. The environment moves to a new state $S_{t+1}$ and the reward $R_{t+1}$ associated with the *transition* $(S_t, A_t, S_{t+1})$ is determined. The goal of a reinforcement learning agent is to learn a *policy*:

$$\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$$
$$\pi(s, a) = \Pr(A_t=a \mid S_t=s)$$

that maximizes the expected cumulative reward.

Formulating the problem as a Markov decision process assumes the agent directly observes the current environmental state; in this case, the problem is said to have *full observability*. If the agent only has access to a subset of states, or if the observed states are corrupted by noise, the agent is said to have *partial observability*, and formally the problem must be formulated as a partially observable Markov decision process. In both cases, the set of actions available to the agent can be restricted. For example, the state of an account balance could be restricted to be positive; if the current value of the state is 3 and the state transition attempts to reduce the value by 4, the transition will not be allowed.

When the agent's performance is compared to that of an agent that acts optimally, the difference in performance yields the notion of regret. In order to act near optimally, the agent must reason about long-term consequences of its actions (i.e., maximize future rewards), although the immediate reward associated with this might be negative.

Thus, reinforcement learning is particularly well-suited to problems that include a long-term versus short-term reward trade-off. It has been applied successfully to various problems, including energy storage,[7] robot control,[8] photovoltaic generators,[9] backgammon, checkers,[10] Go (AlphaGo), and autonomous driving systems.[11]

Two elements make reinforcement learning powerful: the use of samples to optimize performance, and the use of function approximation to deal with large environments. Thanks to these two key components, RL can be used in large environments in the following situations:

- A model of the environment is known, but an analytic solution is not available;
- Only a simulation model of the environment is given (the subject of simulation-based optimization);[12]

- The only way to collect information about the environment is to interact with it.

The first two of these problems could be considered planning problems (since some form of model is available), while the last one could be considered to be a genuine learning problem. However, reinforcement learning converts both planning problems to machine learning problems.

# Exploration

The trade-off between exploration and exploitation has been most thoroughly studied through the multi-armed bandit problem and for finite state space Markov decision processes in Burnetas and Katehakis (1997).[13]

Reinforcement learning requires clever exploration mechanisms; randomly selecting actions, without reference to an estimated probability distribution, shows poor performance. The case of (small) finite Markov decision processes is relatively well understood. However, due to the lack of algorithms that scale well with the number of states (or scale to problems with infinite state spaces), simple exploration methods are the most practical.

One such method is $\varepsilon$-greedy, where $0 < \varepsilon < 1$ is a parameter controlling the amount of exploration vs. exploitation. With probability $1 - \varepsilon$, exploitation is chosen, and the agent chooses the action that it believes has the best long-term effect (ties between actions are broken uniformly at random). Alternatively, with probability $\varepsilon$, exploration is chosen, and the action is chosen uniformly at random. $\varepsilon$ is usually a fixed parameter but can be adjusted either according to a schedule (making the agent explore progressively less), or adaptively based on heuristics.[14]

# Algorithms for control learning

Even if the issue of exploration is disregarded and even if the state was observable (assumed hereafter), the problem remains to use past experience to find out which actions lead to higher cumulative rewards.

## Criterion of optimality

### Policy

The agent's action selection is modeled as a map called *policy*:

$$\pi : \mathcal{A} \times \mathcal{S} \to [0, 1]$$
$$\pi(a, s) = \Pr(A_t = a \mid S_t = s)$$

The policy map gives the probability of taking action $a$ when in state $s$.[15]:61 There are also deterministic policies $\pi$ for which $\pi(s)$ denotes the action that should be played at state $s$.

### State-value function

The state-value function $V_\pi(s)$ is defined as, *expected discounted return* starting with state $s$, i.e. $S_0 = s$, and successively following policy $\pi$. Hence, roughly speaking, the value function estimates "how good" it is to be in a given state.[15]:60

$$V_\pi(s) = \mathbb{E}[G \mid S_0 = s] = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s\right],$$

where the random variable $G$ denotes the **discounted return**, and is defined as the sum of future discounted rewards:

$$G = \sum_{t=0}^{\infty} \gamma^t R_{t+1} = R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots,$$

where $R_{t+1}$ is the reward for transitioning from state $S_t$ to $S_{t+1}$, $0 \leq \gamma < 1$ is the discount rate. $\gamma$ is less than 1, so rewards in the distant future are weighted less than rewards in the immediate future.

The goal is to find a policy that maximizes the expected discounted return. From the theory of Markov decision processes it is known that, without loss of generality, the search can be restricted to the set of so-called *stationary* policies. A policy is *stationary* if the action-distribution returned by it depends only on the last state visited (from the observation agent's history). The search can be further restricted to *deterministic* stationary policies. A *deterministic stationary* policy deterministically selects actions based on the current state. Since any such policy can be identified with a mapping from the set of states to the set of actions, these policies can be identified with such mappings with no loss of generality.

## Brute force

The brute force approach entails two steps:

- For each possible policy, sample returns while following it
- Choose the policy with the largest expected discounted return

One problem with this is that the number of policies can be large, or even infinite. Another is that the variance of the returns may be large, which requires many samples to accurately estimate the discounted return of each policy.

These problems can be ameliorated if we assume some structure and allow samples generated from one policy to influence the estimates made for others. The two main approaches for achieving this are value function estimation and direct policy search.

## Value function

Value function approaches attempt to find a policy that maximizes the discounted return by maintaining a set of estimates of expected discounted returns $\mathbb{E}[G]$ for some policy (usually either the "current" [on-policy] or the optimal [off-policy] one).

These methods rely on the theory of Markov decision processes, where optimality is defined in a sense stronger than the one above: A policy is optimal if it achieves the best-expected discounted return from *any* initial state (i.e., initial distributions play no role in this definition). Again, an optimal policy can always be found among stationary policies.

To define optimality in a formal manner, define the state-value of a policy $\pi$ by

$$V^{\pi}(s) = \mathbb{E}[G \mid s, \pi],$$

where $G$ stands for the discounted return associated with following $\pi$ from the initial state $s$. Defining $V^{*}(s)$ as the maximum possible state-value of $V^{\pi}(s)$, where $\pi$ is allowed to change,

$$V^{*}(s) = \max_{\pi} V^{\pi}(s).$$

A policy that achieves these optimal state-values in each state is called *optimal*. Clearly, a policy that is optimal in this sense is also optimal in the sense that it maximizes the expected discounted return, since $V^{*}(s) = \max_{\pi} \mathbb{E}[G \mid s, \pi]$, where $s$ is a state randomly sampled from the distribution $\mu$ of initial states (so $\mu(s) = \Pr(S_0 = s)$).

Although state-values suffice to define optimality, it is useful to define action-values. Given a state $s$, an action $a$ and a policy $\pi$, the action-value of the pair $(s, a)$ under $\pi$ is defined by

$$Q^{\pi}(s, a) = \mathbb{E}[G \mid s, a, \pi],$$

where $G$ now stands for the random discounted return associated with first taking action $a$ in state $s$ and following $\pi$, thereafter.

The theory of Markov decision processes states that if $\pi^{*}$ is an optimal policy, we act optimally (take the optimal action) by choosing the action from $Q^{\pi^{*}}(s, \cdot)$ with the highest action-value at each state, $s$. The *action-value function* of such an optimal policy ($Q^{\pi^{*}}$) is called the *optimal action-value function* and is commonly denoted by $Q^{*}$. In summary, the knowledge of the optimal action-value function alone suffices to know how to act optimally.

Assuming full knowledge of the Markov decision process, the two basic approaches to compute the optimal action-value function are value iteration and policy iteration. Both algorithms compute a sequence of functions $Q_k$ ($k = 0, 1, 2, \ldots$) that converge to $Q^{*}$. Computing these functions involves computing expectations over the whole state-space, which is impractical for all but the smallest (finite) Markov decision processes. In reinforcement learning methods, expectations are approximated by averaging over samples and using function approximation techniques to cope with the need to represent value functions over large state-action spaces.

### Monte Carlo methods

Monte Carlo methods[16] are used to solve reinforcement learning problems by averaging sample returns. Unlike methods that require full knowledge of the environment's dynamics, Monte Carlo methods rely solely on actual or simulated experience—sequences of states, actions, and rewards obtained from

interaction with an environment. This makes them applicable in situations where the complete dynamics are unknown. Learning from actual experience does not require prior knowledge of the environment and can still lead to optimal behavior. When using simulated experience, only a model capable of generating sample transitions is required, rather than a full specification of transition probabilities, which is necessary for dynamic programming methods.

Monte Carlo methods apply to episodic tasks, where experience is divided into episodes that eventually terminate. Policy and value function updates occur only after the completion of an episode, making these methods incremental on an episode-by-episode basis, though not on a step-by-step (online) basis. The term "Monte Carlo" generally refers to any method involving random sampling; however, in this context, it specifically refers to methods that compute averages from *complete* returns, rather than *partial* returns.

These methods function similarly to the bandit algorithms, in which returns are averaged for each state-action pair. The key difference is that actions taken in one state affect the returns of subsequent states within the same episode, making the problem non-stationary. To address this non-stationarity, Monte Carlo methods use the framework of general policy iteration (GPI). While dynamic programming computes value functions using full knowledge of the Markov decision process, Monte Carlo methods learn these functions through sample returns. The value functions and policies interact similarly to dynamic programming to achieve optimality, first addressing the prediction problem and then extending to policy improvement and control, all based on sampled experience.[15]

### Temporal difference methods

The first problem is corrected by allowing the procedure to change the policy (at some or all states) before the values settle. This too may be problematic as it might prevent convergence. Most current algorithms do this, giving rise to the class of *generalized policy iteration* algorithms. Many *actor-critic* methods belong to this category.

The second issue can be corrected by allowing trajectories to contribute to any state-action pair in them. This may also help to some extent with the third problem, although a better solution when returns have high variance is Sutton's temporal difference (TD) methods that are based on the recursive Bellman equation.[17][18] The computation in TD methods can be incremental (when after each transition the memory is changed and the transition is thrown away), or batch (when the transitions are batched and the estimates are computed once based on the batch). Batch methods, such as the least-squares temporal difference method,[19] may use the information in the samples better, while incremental methods are the only choice when batch methods are infeasible due to their high computational or memory complexity. Some methods try to combine the two approaches. Methods based on temporal differences also overcome the fourth issue.

Another problem specific to TD comes from their reliance on the recursive Bellman equation. Most TD methods have a so-called $\lambda$ parameter $(0 \leq \lambda \leq 1)$ that can continuously interpolate between Monte Carlo methods that do not rely on the Bellman equations and the basic TD methods that rely entirely on the Bellman equations. This can mitigate this issue.

### Function approximation methods

In order to address the fifth issue, *function approximation methods* are used. *Linear function approximation* starts with a mapping $\phi$ that assigns a finite-dimensional vector to each state-action pair. Then, the action values of a state-action pair $(s, a)$ are obtained by linearly combining the components of $\phi(s, a)$ with some *weights* $\theta$:

$$Q(s, a) = \sum_{i=1}^{d} \theta_i \phi_i(s, a).$$

The algorithms then adjust the weights, instead of adjusting the values associated with the individual state-action pairs. Methods based on ideas from nonparametric statistics (which can be seen to construct their own features) have been explored.

Value iteration can also be used as a starting point, giving rise to the Q-learning algorithm and its many variants.[20] Including Deep Q-learning methods when a neural network is used to represent Q, with various applications in stochastic search problems.[21]

The problem with using action-values is that they may need highly precise estimates of the competing action values that can be hard to obtain when the returns are noisy, though this problem is mitigated to some extent by temporal difference methods. Using the so-called compatible function approximation method compromises generality and efficiency.

## Direct policy search

An alternative method is to search directly in (some subset of) the policy space, in which case the problem becomes a case of stochastic optimization. The two approaches available are gradient-based and gradient-free methods.

Gradient-based methods (*policy gradient methods*) start with a mapping from a finite-dimensional (parameter) space to the space of policies: given the parameter vector $\theta$, let $\pi_\theta$ denote the policy associated to $\theta$. Defining the performance function by $\rho(\theta) = \rho^{\pi_\theta}$ under mild conditions this function will be differentiable as a function of the parameter vector $\theta$. If the gradient of $\rho$ was known, one could use gradient ascent. Since an analytic expression for the gradient is not available, only a noisy estimate is available. Such an estimate can be constructed in many ways, giving rise to algorithms such as Williams's REINFORCE method[22] (which is known as the likelihood ratio method in the simulation-based optimization literature).[23]

A large class of methods avoids relying on gradient information. These include simulated annealing, cross-entropy search or methods of evolutionary computation. Many gradient-free methods can achieve (in theory and in the limit) a global optimum.

Policy search methods may converge slowly given noisy data. For example, this happens in episodic problems when the trajectories are long and the variance of the returns is large. Value-function based methods that rely on temporal differences might help in this case. In recent years, *actor–critic methods* have been proposed and performed well on various problems.[24]

Policy search methods have been used in the robotics context.[25] Many policy search methods may get stuck in local optima (as they are based on local search).

### Model-based algorithms

Finally, all of the above methods can be combined with algorithms that first learn a model of the Markov decision process, the probability of each next state given an action taken from an existing state. For instance, the Dyna algorithm learns a model from experience, and uses that to provide more modelled transitions for a value function, in addition to the real transitions.[26] Such methods can sometimes be extended to use of non-parametric models, such as when the transitions are simply stored and "replayed" to the learning algorithm.[27]

Model-based methods can be more computationally intensive than model-free approaches, and their utility can be limited by the extent to which the Markov decision process can be learnt.[28]

There are other ways to use models than to update a value function.[29] For instance, in model predictive control the model is used to update the behavior directly.

### Partially supervised RL (PSRL)

The costly exploration needed to learn an optimal policy can be reduced if some supervised data is available. This can be achieved for instance by learning a crude control policy and using this to initialize the Q table intelligently instead of with zeros.[30]

## Theory

Both the asymptotic and finite-sample behaviors of most algorithms are well understood. Algorithms with provably (i.e. in a way that can be proved) good online performance (addressing the exploration issue) are known.

Efficient exploration of Markov decision processes is given in Burnetas and Katehakis (1997).[13] Finite-time performance bounds have also appeared for many algorithms, but these bounds are expected to be rather loose and thus more work is needed to better understand the relative advantages and limitations.

For incremental algorithms, asymptotic convergence issues have been settled. Temporal-difference-based algorithms converge under a wider set of conditions than was previously possible (for example, when used with arbitrary, smooth function approximation).

## Research

Research topics include:

- actor-critic architecture[31]
- actor-critic-scenery architecture[4]
- adaptive methods that work with fewer (or no) parameters under a large number of conditions

- bug detection in software projects[32]
- continuous learning
- combinations with logic-based frameworks (e.g., temporal-logic specifications,[33] reward machines,[34] and probabilistic argumentation).[35]
- exploration in large Markov decision processes
- entity-based reinforcement learning[36][37][38]
- human feedback[39]
- interaction between implicit and explicit learning in skill acquisition
- intrinsic motivation which differentiates information-seeking, curiosity-type behaviours from task-dependent goal-directed behaviours large-scale empirical evaluations
- large (or continuous) action spaces
- modular and hierarchical reinforcement learning[40]
- multiagent/distributed reinforcement learning is a topic of interest. Applications are expanding.[41]
- occupant-centric control
- optimization of computing resources[42][43][44]
- partial information (e.g., using predictive state representation)
- reward function based on maximising novel information[45][46][47]
- sample-based planning (e.g., based on Monte Carlo tree search).
- securities trading[48]
- transfer learning[49]
- TD learning modeling dopamine-based learning in the brain. Dopaminergic projections from the substantia nigra to the basal ganglia function are the prediction error.
- value-function and policy search methods

# Comparison of key algorithms

The following table lists the key algorithms for learning a policy depending on several criteria:

- The algorithm can be on-policy (it performs policy updates using trajectories sampled via the current policy)[50] or off-policy.
- The action space may be discrete (e.g. the action space could be "going up", "going left", "going right", "going down", "stay") or continuous (e.g. moving the arm with a given angle).
- The state space may be discrete (e.g. the agent could be in a cell in a grid) or continuous (e.g. the agent could be located at a given position in the plane).

| Algorithm | Description | Policy | Action space | State space | Operator |
|---|---|---|---|---|---|
| Monte Carlo | Every visit to Monte Carlo | Either | Discrete | Discrete | Sample-means of state-values or action-values |
| TD learning | State–action–reward–state | Off-policy | Discrete | Discrete | State-value |
| Q-learning | State–action–reward–state | Off-policy | Discrete | Discrete | Action-value |
| SARSA | State–action–reward–state–action | On-policy | Discrete | Discrete | Action-value |
| DQN | Deep Q Network | Off-policy | Discrete | Continuous | Action-value |
| DDPG | Deep Deterministic Policy Gradient | Off-policy | Continuous | Continuous | Action-value |
| A3C | Asynchronous Advantage Actor-Critic Algorithm | On-policy | Continuous[51] or Discrete | Continuous | Advantage (=action-value - state-value) |
| TRPO | Trust Region Policy Optimization | On-policy | Continuous or Discrete | Continuous | Advantage |
| PPO | Proximal Policy Optimization | On-policy | Continuous or Discrete | Continuous | Advantage |
| TD3 | Twin Delayed Deep Deterministic Policy Gradient | Off-policy | Continuous | Continuous | Action-value |
| SAC | Soft Actor-Critic | Off-policy | Continuous | Continuous | Advantage |
| DSAC[52][53][54] | Distributional Soft Actor Critic | Off-policy | Continuous | Continuous | Action-value distribution |

## Associative reinforcement learning

Associative reinforcement learning tasks combine facets of stochastic learning automata tasks and supervised learning pattern classification tasks. In associative reinforcement learning tasks, the learning system interacts in a closed loop with its environment.[55]

## Deep reinforcement learning

This approach extends reinforcement learning by using a deep neural network and without explicitly designing the state space.[56] The work on learning ATARI games by Google DeepMind increased attention to deep reinforcement learning or end-to-end reinforcement learning.[57]

## Adversarial deep reinforcement learning

Adversarial deep reinforcement learning is an active area of research in reinforcement learning focusing on vulnerabilities of learned policies. In this research area some studies initially showed that reinforcement learning policies are susceptible to imperceptible adversarial manipulations.[58][59][60]

While some methods have been proposed to overcome these susceptibilities, in the most recent studies it has been shown that these proposed solutions are far from providing an accurate representation of current vulnerabilities of deep reinforcement learning policies.[61]

## Fuzzy reinforcement learning

By introducing fuzzy inference in reinforcement learning,[62] approximating the state-action value function with fuzzy rules in continuous space becomes possible. The IF - THEN form of fuzzy rules make this approach suitable for expressing the results in a form close to natural language. Extending FRL with Fuzzy Rule Interpolation[63] allows the use of reduced size sparse fuzzy rule-bases to emphasize cardinal rules (most important state-action values).

## Inverse reinforcement learning

In inverse reinforcement learning (IRL), no reward function is given. Instead, the reward function is inferred given an observed behavior from an expert. The idea is to mimic observed behavior, which is often optimal or close to optimal.[64] One popular IRL paradigm is named maximum entropy inverse reinforcement learning (MaxEnt IRL).[65] MaxEnt IRL estimates the parameters of a linear model of the reward function by maximizing the entropy of the probability distribution of observed trajectories subject to constraints related to matching expected feature counts. Recently it has been shown that MaxEnt IRL is a particular case of a more general framework named random utility inverse reinforcement learning (RU-IRL).[66] RU-IRL is based on random utility theory and Markov decision processes. While prior IRL approaches assume that the apparent random behavior of an observed agent is due to it following a random policy, RU-IRL assumes that the observed agent follows a deterministic policy but randomness in observed behavior is due to the fact that an observer only has partial access to the features the observed agent uses in decision making. The utility function is modeled as a random variable to account for the ignorance of the observer regarding the features the observed agent actually considers in its utility function.

## Multi-objective reinforcement learning

Multi-objective reinforcement learning (MORL) is a form of reinforcement learning concerned with conflicting alternatives. It is distinct from multi-objective optimization in that it is concerned with agents acting in environments.[67][68]

## Safe reinforcement learning

Safe reinforcement learning (SRL) can be defined as the process of learning policies that maximize the expectation of the return in problems in which it is important to ensure reasonable system performance and/or respect safety constraints during the learning and/or deployment processes.[69][70] An alternative approach is risk-averse reinforcement learning, where instead of the *expected* return, a *risk-measure* of the return is optimized, such as the conditional value at risk (CVaR).[71] In addition to mitigating risk, the CVaR objective increases robustness to model uncertainties.[72][73] However, CVaR optimization in risk-averse RL requires special care, to prevent gradient bias[74] and blindness to success.[75]

### Self-reinforcement learning

Self-reinforcement learning (or self-learning), is a learning paradigm which does not use the concept of immediate reward $R_a(s, s')$ after transition from $s$ to $s'$ with action $a$. It does not use an external reinforcement, it only uses the agent internal self-reinforcement. The internal self-reinforcement is provided by mechanism of feelings and emotions. In the learning process emotions are backpropagated by a mechanism of secondary reinforcement. The learning equation does not include the immediate reward, it only includes the state evaluation.

The self-reinforcement algorithm updates a memory matrix $W = \|w(a, s)\|$ such that in each iteration executes the following machine learning routine:

1. In situation $s$ perform action $a$.
2. Receive a consequence situation $s'$.
3. Compute state evaluation $v(s')$ of how good it is to be in the consequence situation $s'$.
4. Update crossbar memory $w'(a, s) = w(a, s) + v(s')$.

Initial conditions of the memory are received as input from the genetic environment. It is a system with only one input (situation), and only one output (action, or behavior).

Self-reinforcement (self-learning) was introduced in 1982 along with a neural network capable of self-reinforcement learning, named Crossbar Adaptive Array (CAA).[76][77] The CAA computes, in a crossbar fashion, both decisions about actions and emotions (feelings) about consequence states. The system is driven by the interaction between cognition and emotion.[78]

## Statistical comparison of reinforcement learning algorithms

Efficient comparison of RL algorithms is essential for research, deployment and monitoring of RL systems. To compare different algorithms on a given environment, an agent can be trained for each algorithm. Since the performance is sensitive to implementation details, all algorithms should be implemented as closely as possible to each other.[79] After the training is finished, the agents can be run on a sample of test episodes, and their scores (returns) can be compared. Since episodes are typically assumed to be i.i.d, standard statistical tools can be used for hypothesis testing, such as T-test and permutation test.[80] This requires to accumulate all the rewards within an episode into a single number—the episodic return. However, this causes a loss of information, as different time-steps are averaged together, possibly with different levels of noise. Whenever the noise level varies across the episode, the statistical power can be improved significantly, by weighting the rewards according to their estimated noise.[81]

## Challenges and limitations

Despite significant advancements, reinforcement learning (RL) continues to face several challenges and limitations that hinder its widespread application in real-world scenarios.

### Sample inefficiency

RL algorithms often require a large number of interactions with the environment to learn effective policies, leading to high computational costs and time-intensive to train the agent. For instance, OpenAI's Dota-playing bot utilized thousands of years of simulated gameplay to achieve human-level performance. Techniques like experience replay and curriculum learning have been proposed to reduce sample inefficiency, but these techniques add more complexity and are not always sufficient for real-world applications.

### Stability and convergence issues

Training RL models, particularly for deep neural network-based models, can be unstable and prone to divergence. A small change in the policy or environment can lead to extreme fluctuations in performance, making it difficult to achieve consistent results. This instability is further enhanced in the case of the continuous or high-dimensional action space, where the learning step becomes more complex and less predictable.

### Generalization and transferability

The RL agents trained in specific environments often struggle to generalize their learned policies to new, unseen scenarios. This is the major setback preventing the application of RL to dynamic real-world environments where adaptability is crucial. The challenge is to develop such algorithms that can transfer knowledge across tasks and environments without extensive retraining.

### Bias and reward function issues

Designing appropriate reward functions is critical in RL because poorly designed reward functions can lead to unintended behaviors. In addition, RL systems trained on biased data may perpetuate existing biases and lead to discriminatory or unfair outcomes. Both of these issues requires careful consideration of reward structures and data sources to ensure fairness and desired behaviors.

## In natural language processing

In natural language processing (NLP), reinforcement learning has been applied to tasks in which text generation is treated as a sequential decision problem: the model's policy selects words or utterances as actions, and the reward measures properties of the completed output that are difficult to express as a per-token supervised loss.[82] Early applications used policy-gradient methods such as REINFORCE to optimize sequence-level evaluation metrics, including BLEU in machine translation and ROUGE in text summarization,[83][84] and to train dialogue systems.[85]

Reinforcement learning from human feedback (RLHF) is used to train large language models. In RLHF, human annotators compare or rank model outputs, a reward model is trained on these preference judgments, and the language model is then fine-tuned with a policy-gradient algorithm, commonly proximal policy optimization (PPO), to produce outputs that the reward model scores highly.[86][87] The reward model substitutes for human raters during optimization, so new human judgments are not required at each training step, although the method as a whole depends on a dataset of human-annotated preferences. OpenAI used RLHF to train InstructGPT, released in January 2022, and ChatGPT, released

in November 2022.[88] Direct preference optimization (DPO), published in 2023, instead fine-tunes the language model on the preference data directly, without a separate reward model or reinforcement learning loop.[89]

Later work on language models designed for multi-step reasoning replaced the learned reward model with rewards computed automatically from the task, such as checking a mathematical answer against a reference solution or running unit tests on generated code.[90] OpenAI o1, released in 2024, and DeepSeek-R1, released in 2025, were trained with large-scale reinforcement learning of this kind to produce a chain of thought before answering.[91][92] DeepSeek-R1's developers reported that reasoning behaviors such as self-verification and reflection emerged when reinforcement learning was applied directly to a pretrained base model, without a preliminary supervised fine-tuning stage.[92]

## See also

- Active learning (machine learning)
- Apprenticeship learning
- Error-driven learning
- Model-free (reinforcement learning)
- Multi-agent reinforcement learning
- Optimal control
- Q-learning
- Reinforcement learning from human feedback
- State–action–reward–state–action (SARSA)
- Temporal difference learning

## References


1. Wu, Cathy. "Dynamic programming: What makes sequential decision making hard?" (https://web.mit.edu/6.7950/www/lectures/L1-2022fa.pdf) (PDF). p. 6.
2. Kaelbling, Leslie P.; Littman, Michael L.; Moore, Andrew W. (1996). "Reinforcement Learning: A Survey" (http://webarchive.loc.gov/all/20011120234539/http://www.cs.washington.edu/research/jair/abstracts/kaelbling96a.html). *Journal of Artificial Intelligence Research*. **4**: 237–285. arXiv:cs/9605103 (https://arxiv.org/abs/cs/9605103). doi:10.1613/jair.301 (https://doi.org/10.1613%2Fjair.301). S2CID 1708582 (https://api.semanticscholar.org/CorpusID:1708582). Archived from the original (http://www.cs.washington.edu/research/jair/abstracts/kaelbling96a.html) on 2001-11-20.
3. van Otterlo, M.; Wiering, M. (2012). "Reinforcement Learning and Markov Decision Processes". *Reinforcement Learning*. Adaptation, Learning, and Optimization. Vol. 12. pp. 3–42. doi:10.1007/978-3-642-27645-3_1 (https://doi.org/10.1007%2F978-3-642-27645-3_1). ISBN 978-3-642-27644-6.
4. Li, Shengbo (2023). *Reinforcement Learning for Sequential Decision and Optimal Control* (https://link.springer.com/book/10.1007/978-981-19-7784-8) (First ed.). Springer Verlag, Singapore. pp. 1–460. doi:10.1007/978-981-19-7784-8 (https://doi.org/10.1007%2F978-981-19-7784-8). ISBN 978-9-811-97783-1. S2CID 257928563 (https://api.semanticscholar.org/CorpusID:257928563).
5. Russell, Stuart J.; Norvig, Peter (2010). *Artificial intelligence: a modern approach* (Third ed.). Upper Saddle River, New Jersey: Prentice Hall. pp. 830, 831. ISBN 978-0-13-604259-4.

6. Lee, Daeyeol; Seo, Hyojung; Jung, Min Whan (21 July 2012). "Neural Basis of Reinforcement Learning and Decision Making" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3490621). *Annual Review of Neuroscience*. **35** (1): 287–308. doi:10.1146/annurev-neuro-062111-150512 (https://doi.org/10.1146%2Fannurev-neuro-062111-150512). PMC 3490621 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3490621). PMID 22462543 (https://pubmed.ncbi.nlm.nih.gov/22462543).
7. Salazar Duque, Edgar Mauricio; Giraldo, Juan S.; Vergara, Pedro P.; Nguyen, Phuong; Van Der Molen, Anne; Slootweg, Han (2022). "Community energy storage operation via reinforcement learning with eligibility traces" (https://doi.org/10.1016%2Fj.epsr.2022.108515). *Electric Power Systems Research*. **212** 108515. Bibcode:2022EPSR..21208515S (https://ui.adsabs.harvard.edu/abs/2022EPSR..21208515S). doi:10.1016/j.epsr.2022.108515 (https://doi.org/10.1016%2Fj.epsr.2022.108515). S2CID 250635151 (https://api.semanticscholar.org/CorpusID:250635151).
8. Xie, Zhaoming; Hung Yu Ling; Nam Hee Kim; Michiel van de Panne (2020). "ALLSTEPS: Curriculum-driven Learning of Stepping Stone Skills". arXiv:2005.04323 (https://arxiv.org/abs/2005.04323) [cs.GR (https://arxiv.org/archive/cs.GR)].
9. Vergara, Pedro P.; Salazar, Mauricio; Giraldo, Juan S.; Palensky, Peter (2022). "Optimal dispatch of PV inverters in unbalanced distribution systems using Reinforcement Learning" (https://doi.org/10.1016%2Fj.ijepes.2021.107628). *International Journal of Electrical Power & Energy Systems*. **136** 107628. Bibcode:2022IJEPE.13607628V (https://ui.adsabs.harvard.edu/abs/2022IJEPE.13607628V). doi:10.1016/j.ijepes.2021.107628 (https://doi.org/10.1016%2Fj.ijepes.2021.107628). S2CID 244099841 (https://api.semanticscholar.org/CorpusID:244099841).
10. Sutton & Barto 2018, Chapter 11.
11. Ren, Yangang; Jiang, Jianhua; Zhan, Guojian; Li, Shengbo Eben; Chen, Chen; Li, Keqiang; Duan, Jingliang (2026). "Self-Learned Intelligence for Integrated Decision and Control of Automated Vehicles at Signalized Intersections". *IEEE Transactions on Intelligent Transportation Systems*. **23** (12): 24145–24156. arXiv:2110.12359 (https://arxiv.org/abs/2110.12359). Bibcode:2022ITITr..2324145R (https://ui.adsabs.harvard.edu/abs/2022ITITr..2324145R). doi:10.1109/TITS.2022.3196167 (https://doi.org/10.1109%2FTITS.2022.3196167).
12. Gosavi, Abhijit (2003). *Simulation-based Optimization: Parametric Optimization Techniques and Reinforcement* (https://www.springer.com/mathematics/applications/book/978-1-4020-7454-7). Operations Research/Computer Science Interfaces Series. Springer. ISBN 978-1-4020-7454-7.
13. Burnetas, Apostolos N.; Katehakis, Michael N. (1997), "Optimal adaptive policies for Markov Decision Processes", *Mathematics of Operations Research*, **22** (1): 222–255, doi:10.1287/moor.22.1.222 (https://doi.org/10.1287%2Fmoor.22.1.222), JSTOR 3690147 (https://www.jstor.org/stable/3690147)
14. Tokic, Michel; Palm, Günther (2011), "Value-Difference Based Exploration: Adaptive Control Between Epsilon-Greedy and Softmax" (http://www.tokic.com/www/tokicm/publikationen/papers/KI2011.pdf) (PDF), *KI 2011: Advances in Artificial Intelligence*, Lecture Notes in Computer Science, vol. 7006, Springer, pp. 335–346, ISBN 978-3-642-24455-1
15. "Reinforcement learning: An introduction" (https://web.archive.org/web/20170712170739/http://people.inf.elte.hu/lorincz/Files/RL_2006/SuttonBook.pdf) (PDF). Archived from the original (http://people.inf.elte.hu/lorincz/Files/RL_2006/SuttonBook.pdf) (PDF) on 2017-07-12. Retrieved 2017-07-23.
16. Singh, Satinder P.; Sutton, Richard S. (1996-03-01). "Reinforcement learning with replacing eligibility traces" (https://link.springer.com/article/10.1007/BF00114726). *Machine Learning*. **22** (1): 123–158. doi:10.1007/BF00114726 (https://doi.org/10.1007%2FBF00114726). ISSN 1573-0565 (https://search.worldcat.org/issn/1573-0565).

17. Sutton, Richard S. (1984). *Temporal Credit Assignment in Reinforcement Learning* (https://web.archive.org/web/20170330002227/http://incompleteideas.net/sutton/publications.html#PhDthesis) (PhD thesis). University of Massachusetts, Amherst, MA. Archived from the original (http://incompleteideas.net/sutton/publications.html#PhDthesis) on 2017-03-30. Retrieved 2017-03-29.
18. Sutton & Barto 2018, §6. Temporal-Difference Learning (http://incompleteideas.net/sutton/book/ebook/node60.html).
19. Bradtke, Steven J.; Barto, Andrew G. (1996). "Learning to predict by the method of temporal differences". *Machine Learning*. **22**: 33–57. CiteSeerX 10.1.1.143.857 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.143.857). doi:10.1023/A:1018056104778 (https://doi.org/10.1023%2FA%3A1018056104778). S2CID 20327856 (https://api.semanticscholar.org/CorpusID:20327856). {{cite journal}}: Cite uses deprecated parameter |citeseerx= (help)
20. Watkins, Christopher J.C.H. (1989). *Learning from Delayed Rewards* (http://www.cs.rhul.ac.uk/~chrisw/new_thesis.pdf) (PDF) (PhD thesis). King's College, Cambridge, UK.
21. Matzliach, Barouch; Ben-Gal, Irad; Kagan, Evgeny (2022). "Detection of Static and Mobile Targets by an Autonomous Agent with Deep Q-Learning Abilities" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9407070). *Entropy*. **24** (8): 1168. Bibcode:2022Entrp..24.1168M (https://ui.adsabs.harvard.edu/abs/2022Entrp..24.1168M). doi:10.3390/e24081168 (https://doi.org/10.3390%2Fe24081168). PMC 9407070 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9407070). PMID 36010832 (https://pubmed.ncbi.nlm.nih.gov/36010832).
22. Williams, Ronald J. (1987). "A class of gradient-estimating algorithms for reinforcement learning in neural networks". *Proceedings of the IEEE First International Conference on Neural Networks*. CiteSeerX 10.1.1.129.8871 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.129.8871). {{cite conference}}: Cite uses deprecated parameter |citeseerx= (help)
23. Peters, Jan; Vijayakumar, Sethu; Schaal, Stefan (2003). *Reinforcement Learning for Humanoid Robotics* (https://web.archive.org/web/20130512223911/http://www-clmc.usc.edu/publications/p/peters-ICHR2003.pdf) (PDF). IEEE-RAS International Conference on Humanoid Robots. Archived from the original (http://www-clmc.usc.edu/publications/p/peters-ICHR2003.pdf) (PDF) on 2013-05-12. Retrieved 2006-05-08.
24. Juliani, Arthur (2016-12-17). "Simple Reinforcement Learning with Tensorflow Part 8: Asynchronous Actor-Critic Agents (A3C)" (https://medium.com/emergent-future/simple-reinforcement-learning-with-tensorflow-part-8-asynchronous-actor-critic-agents-a3c-c88f72a5e9f2). *Medium*. Retrieved 2018-02-22.
25. Deisenroth, Marc Peter; Neumann, Gerhard; Peters, Jan (2013). *A Survey on Policy Search for Robotics* (http://eprints.lincoln.ac.uk/28029/1/PolicySearchReview.pdf) (PDF). Foundations and Trends in Robotics. Vol. 2. NOW Publishers. pp. 1–142. doi:10.1561/2300000021 (https://doi.org/10.1561%2F2300000021). hdl:10044/1/12051 (https://hdl.handle.net/10044%2F1%2F12051).
26. Sutton, Richard (1990). "Integrated Architectures for Learning, Planning and Reacting based on Dynamic Programming". *Machine Learning: Proceedings of the Seventh International Workshop*.
27. Lin, Long-Ji (1992). "Self-improving reactive agents based on reinforcement learning, planning and teaching" (https://link.springer.com/content/pdf/10.1007/BF00992699.pdf) (PDF). *Machine Learning*. Vol. 8. doi:10.1007/BF00992699 (https://doi.org/10.1007%2FBF00992699).
28. Zou, Lan (2023-01-01), "Chapter 7 - Meta-reinforcement learning" (https://www.sciencedirect.com/science/article/pii/B9780323899314000110), in Zou, Lan (ed.), *Meta-Learning*, Academic Press, pp. 267–297, doi:10.1016/b978-0-323-89931-4.00011-0 (https://doi.org/10.1016%2Fb978-0-323-89931-4.00011-0), ISBN 978-0-323-89931-4, retrieved 2023-11-08

29. van Hasselt, Hado; Hessel, Matteo; Aslanides, John (2019). "When to use parametric models in reinforcement learning?" (https://proceedings.neurips.cc/paper/2019/file/1b742ae215adf18b75449c6e272fd92d-Paper.pdf) (PDF). *Advances in Neural Information Processing Systems*. Vol. 32.
30. Khuat, Thanh Tung; Bassett, Robert; Otte, Ellen; Grevis-James, Alistair; Gabrys, Bogdan (2024-03-01). "Applications of machine learning in antibody discovery, process development, manufacturing and formulation: Current trends, challenges, and opportunities" (https://www.sciencedirect.com/science/article/pii/S0098135424000036). *Computers & Chemical Engineering*. **182** 108585. doi:10.1016/j.compchemeng.2024.108585 (https://doi.org/10.1016%2Fj.compchemeng.2024.108585). ISSN 0098-1354 (https://search.worldcat.org/issn/0098-1354).
31. Grondman, Ivo; Vaandrager, Maarten; Busoniu, Lucian; Babuska, Robert; Schuitema, Erik (2012-06-01). "Efficient Model Learning Methods for Actor–Critic Control" (https://dl.acm.org/doi/10.1109/TSMCB.2011.2170565). *IEEE Transactions on Systems, Man, and Cybernetics - Part B: Cybernetics*. **42** (3): 591–602. Bibcode:2012ITSMC..42..591G (https://ui.adsabs.harvard.edu/abs/2012ITSMC..42..591G). doi:10.1109/TSMCB.2011.2170565 (https://doi.org/10.1109%2FTSMCB.2011.2170565). ISSN 1083-4419 (https://search.worldcat.org/issn/1083-4419). PMID 22156998 (https://pubmed.ncbi.nlm.nih.gov/22156998).
32. "On the Use of Reinforcement Learning for Testing Game Mechanics: ACM - Computers in Entertainment" (https://cie.acm.org/articles/use-reinforcements-learning-testing-game-mechanics/). *cie.acm.org*. Retrieved 2018-11-27.
33. Li, Xiao; Vasile, Cristian-Ioan; Belta, Calin (2017). "Reinforcement Learning with Temporal Logic Rewards" (https://dl.acm.org/doi/10.1109/IROS.2017.8206234). *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 3834–3839. doi:10.1109/IROS.2017.8206234 (https://doi.org/10.1109%2FIROS.2017.8206234).
34. Toro Icarte, Rodrigo; Klassen, Toryn Q.; Valenzano, Richard; McIlraith, Sheila A. (2022). "Reward Machines: Exploiting Reward Function Structure in Reinforcement Learning" (https://dl.acm.org/doi/10.1613/jair.1.12440). *Journal of Artificial Intelligence Research*. **73**: 173–208. arXiv:2010.03950 (https://arxiv.org/abs/2010.03950). doi:10.1613/jair.1.12440 (https://doi.org/10.1613%2Fjair.1.12440).
35. Riveret, Régis; Gao, Yang; Governatori, Guido; Rotolo, Antonino; Pitt, Jeremy; Sartor, Giovanni (2019). "A probabilistic argumentation framework for reinforcement learning agents" (https://link.springer.com/article/10.1007/s10458-019-09404-2). *Autonomous Agents and Multi-Agent Systems*. **33** (1–2): 216–274. doi:10.1007/s10458-019-09404-2 (https://doi.org/10.1007%2Fs10458-019-09404-2).
36. Haramati, Dan; Daniel, Tal; Tamar, Aviv (2024). "Entity-Centric Reinforcement Learning for Object Manipulation from Pixels". arXiv:2404.01220 (https://arxiv.org/abs/2404.01220) [cs.RO (https://arxiv.org/archive/cs.RO)].
37. Thompson, Isaac Symes; Caron, Alberto; Hicks, Chris; Mavroudis, Vasilios (2024-11-07). "Entity-based Reinforcement Learning for Autonomous Cyber Defence". *Proceedings of the Workshop on Autonomous Cybersecurity (AutonomousCyber '24)*. ACM. pp. 56–67. arXiv:2410.17647 (https://arxiv.org/abs/2410.17647). doi:10.1145/3689933.3690835 (https://doi.org/10.1145%2F3689933.3690835).
38. Winter, Clemens (2023-04-14). "Entity-Based Reinforcement Learning" (https://clemenswinter.com/2023/04/14/entity-based-reinforcement-learning/). *Clemens Winter's Blog*.
39. Yamagata, Taku; McConville, Ryan; Santos-Rodriguez, Raul (2021-11-16). "Reinforcement Learning with Feedback from Multiple Humans with Diverse Skills". arXiv:2111.08596 (https://arxiv.org/abs/2111.08596) [cs.LG (https://arxiv.org/archive/cs.LG)].

40. Kulkarni, Tejas D.; Narasimhan, Karthik R.; Saeedi, Ardavan; Tenenbaum, Joshua B. (2016). "Hierarchical Deep Reinforcement Learning: Integrating Temporal Abstraction and Intrinsic Motivation" (http://dl.acm.org/citation.cfm?id=3157382.3157509). *Proceedings of the 30th International Conference on Neural Information Processing Systems*. NIPS'16. USA: Curran Associates Inc.: 3682–3690. arXiv:1604.06057 (https://arxiv.org/abs/1604.06057). Bibcode:2016arXiv160406057K (https://ui.adsabs.harvard.edu/abs/2016arXiv160406057K). ISBN 978-1-5108-3881-9.
41. "Reinforcement Learning / Successes of Reinforcement Learning" (http://umichrl.pbworks.com/Successes-of-Reinforcement-Learning/). *umichrl.pbworks.com*. Retrieved 2017-08-06.
42. Dey, Somdip; Singh, Amit Kumar; Wang, Xiaohang; McDonald-Maier, Klaus (March 2020). "User Interaction Aware Reinforcement Learning for Power and Thermal Efficiency of CPU-GPU Mobile MPSoCs" (https://ieeexplore.ieee.org/document/9116294). *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)* (http://repository.essex.ac.uk/27546/1/User%20Interaction%20Aware%20Reinforcement%20Learning.pdf) (PDF). pp. 1728–1733. doi:10.23919/DATE48585.2020.9116294 (https://doi.org/10.23919%2FDATE48585.2020.9116294). ISBN 978-3-9819263-4-7. S2CID 219858480 (https://api.semanticscholar.org/CorpusID:219858480).
43. Quested, Tony. "Smartphones get smarter with Essex innovation" (https://www.businessweekly.co.uk/news/academia-research/smartphones-get-smarter-essex-innovation). *Business Weekly*. Retrieved 2021-06-17.
44. Williams, Rhiannon (2020-07-21). "Future smartphones 'will prolong their own battery life by monitoring owners' behaviour' " (https://inews.co.uk/news/technology/future-smartphones-prolong-battery-life-monitoring-behaviour-558689). *i*. Retrieved 2021-06-17.
45. Kaplan, F.; Oudeyer, P. (2004). "Maximizing Learning Progress: An Internal Reward System for Development". In Iida, F.; Pfeifer, R.; Steels, L.; Kuniyoshi, Y. (eds.). *Embodied Artificial Intelligence*. Lecture Notes in Computer Science. Vol. 3139. Berlin; Heidelberg: Springer. pp. 259–270. doi:10.1007/978-3-540-27833-7_19 (https://doi.org/10.1007%2F978-3-540-27833-7_19). ISBN 978-3-540-22484-6. S2CID 9781221 (https://api.semanticscholar.org/CorpusID:9781221).
46. Klyubin, A.; Polani, D.; Nehaniv, C. (2008). "Keep your options open: an information-based driving principle for sensorimotor systems" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2607028). *PLOS ONE*. **3** (12) e4018. Bibcode:2008PLoSO...3.4018K (https://ui.adsabs.harvard.edu/abs/2008PLoSO...3.4018K). doi:10.1371/journal.pone.0004018 (https://doi.org/10.1371%2Fjournal.pone.0004018). PMC 2607028 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2607028). PMID 19107219 (https://pubmed.ncbi.nlm.nih.gov/19107219).
47. Barto, A. G. (2013). "Intrinsic motivation and reinforcement learning". *Intrinsically Motivated Learning in Natural and Artificial Systems* (https://people.cs.umass.edu/~barto/IMCleVer-chapter-totypeset2.pdf) (PDF). Berlin; Heidelberg: Springer. pp. 17–47.
48. Dabérius, Kevin; Granat, Elvin; Karlsson, Patrik (2020). "Deep Execution - Value and Policy Based Reinforcement Learning for Trading and Beating Market Benchmarks". *The Journal of Machine Learning in Finance*. **1**. SSRN 3374766 (https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3374766).
49. George Karimpanal, Thommen; Bouffanais, Roland (2019). "Self-organizing maps for storage and transfer of knowledge in reinforcement learning". *Adaptive Behavior*. **27** (2): 111–126. arXiv:1811.08318 (https://arxiv.org/abs/1811.08318). doi:10.1177/1059712318818568 (https://doi.org/10.1177%2F1059712318818568). ISSN 1059-7123 (https://search.worldcat.org/issn/1059-7123). S2CID 53774629 (https://api.semanticscholar.org/CorpusID:53774629).
50. cf. Sutton & Barto 2018, Section 5.4, p. 100
51. Mnih, Volodymyr; Badia, Adrià; Mirza, Mehdi; Graves, Alex; Lillicrap, Timothy; Harley, Tim; Silver, David; Kavukcuoglu, Koray (16 June 2016). "Asynchronous Methods for Deep Reinforcement Learning" (https://proceedings.mlr.press/v48/mniha16.html). *Proceedings of Machine Learning Research (PMLR)*. **48**. PMLR: 1928–1937. Retrieved 15 June 2026.

52. J Duan; Y Guan; S Li (2021). "Distributional Soft Actor-Critic: Off-policy reinforcement learning for addressing value estimation errors". *IEEE Transactions on Neural Networks and Learning Systems*. **33** (11): 6584–6598. arXiv:2001.02811 (https://arxiv.org/abs/2001.02811). doi:10.1109/TNNLS.2021.3082568 (https://doi.org/10.1109%2FTNNLS.2021.3082568). PMID 34101599 (https://pubmed.ncbi.nlm.nih.gov/34101599). S2CID 211259373 (https://api.semanticscholar.org/CorpusID:211259373).
53. Y Ren; J Duan; S Li (2020). "Improving Generalization of Reinforcement Learning with Minimax Distributional Soft Actor-Critic". *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*. pp. 1–6. arXiv:2002.05502 (https://arxiv.org/abs/2002.05502). doi:10.1109/ITSC45102.2020.9294300 (https://doi.org/10.1109%2FITSC45102.2020.9294300). ISBN 978-1-7281-4149-7. S2CID 211096594 (https://api.semanticscholar.org/CorpusID:211096594).
54. Duan, J; Wang, W; Xiao, L (2025). "Distributional Soft Actor-Critic with Three Refinements". *IEEE Transactions on Pattern Analysis and Machine Intelligence*. **PP** (5): 3935–3946. arXiv:2310.05858 (https://arxiv.org/abs/2310.05858). Bibcode:2025ITPAM..47.3935D (https://ui.adsabs.harvard.edu/abs/2025ITPAM..47.3935D). doi:10.1109/TPAMI.2025.3537087 (https://doi.org/10.1109%2FTPAMI.2025.3537087). PMID 40031258 (https://pubmed.ncbi.nlm.nih.gov/40031258).
55. Soucek, Branko (6 May 1992). *Dynamic, Genetic and Chaotic Programming: The Sixth-Generation Computer Technology Series*. John Wiley & Sons, Inc. p. 38. ISBN 0-471-55717-X.
56. Francois-Lavet, Vincent; et al. (2018). "An Introduction to Deep Reinforcement Learning". *Foundations and Trends in Machine Learning*. **11** (3–4): 219–354. arXiv:1811.12560 (https://arxiv.org/abs/1811.12560). Bibcode:2018arXiv181112560F (https://ui.adsabs.harvard.edu/abs/2018arXiv181112560F). doi:10.1561/2200000071 (https://doi.org/10.1561%2F2200000071). S2CID 54434537 (https://api.semanticscholar.org/CorpusID:54434537).
57. Mnih, Volodymyr; et al. (2015). "Human-level control through deep reinforcement learning". *Nature*. **518** (7540): 529–533. Bibcode:2015Natur.518..529M (https://ui.adsabs.harvard.edu/abs/2015Natur.518..529M). doi:10.1038/nature14236 (https://doi.org/10.1038%2Fnature14236). PMID 25719670 (https://pubmed.ncbi.nlm.nih.gov/25719670). S2CID 205242740 (https://api.semanticscholar.org/CorpusID:205242740).
58. Goodfellow, Ian; Shlens, Jonathan; Szegedy, Christian (2015). "Explaining and Harnessing Adversarial Examples". *International Conference on Learning Representations*. arXiv:1412.6572 (https://arxiv.org/abs/1412.6572).
59. Behzadan, Vahid; Munir, Arslan (2017). "Vulnerability of Deep Reinforcement Learning to Policy Induction Attacks". *Machine Learning and Data Mining in Pattern Recognition*. Lecture Notes in Computer Science. Vol. 10358. pp. 262–275. arXiv:1701.04143 (https://arxiv.org/abs/1701.04143). doi:10.1007/978-3-319-62416-7_19 (https://doi.org/10.1007%2F978-3-319-62416-7_19). ISBN 978-3-319-62415-0. S2CID 1562290 (https://api.semanticscholar.org/CorpusID:1562290).
60. Huang, Sandy; Papernot, Nicolas; Goodfellow, Ian; Duan, Yan; Abbeel, Pieter (2017-02-07). *Adversarial Attacks on Neural Network Policies*. OCLC 1106256905 (https://search.worldcat.org/oclc/1106256905).
61. Korkmaz, Ezgi (2022). "Deep Reinforcement Learning Policies Learn Shared Adversarial Features Across MDPs" (https://doi.org/10.1609%2Faaai.v36i7.20684). *Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)*. **36** (7): 7229–7238. arXiv:2112.09025 (https://arxiv.org/abs/2112.09025). doi:10.1609/aaai.v36i7.20684 (https://doi.org/10.1609%2Faaai.v36i7.20684). S2CID 245219157 (https://api.semanticscholar.org/CorpusID:245219157).
62. Berenji, H.R. (1994). "Fuzzy Q-learning: A new approach for fuzzy dynamic programming". *Proceedings of 1994 IEEE 3rd International Fuzzy Systems Conference*. Orlando, FL, USA: IEEE. pp. 486–491. doi:10.1109/FUZZY.1994.343737 (https://doi.org/10.1109%2FFUZZY.1994.343737). ISBN 0-7803-1896-X. S2CID 56694947 (https://api.semanticscholar.org/CorpusID:56694947).

63. Vincze, David (2017). "Fuzzy rule interpolation and reinforcement learning" (http://users.iit.uni-miskolc.hu/~vinczed/research/vinczed_sami2017_author_draft.pdf) (PDF). *2017 IEEE 15th International Symposium on Applied Machine Intelligence and Informatics (SAMI)*. IEEE. pp. 173–178. doi:10.1109/SAMI.2017.7880298 (https://doi.org/10.1109%2FSAMI.2017.7880298). ISBN 978-1-5090-5655-2. S2CID 17590120 (https://api.semanticscholar.org/CorpusID:17590120).
64. Ng, A. Y.; Russell, S. J. (2000). "Algorithms for Inverse Reinforcement Learning" (https://ai.stanford.edu/~ang/papers/icml00-irl.pdf) (PDF). *Proceeding ICML '00 Proceedings of the Seventeenth International Conference on Machine Learning*. Morgan Kaufmann Publishers. pp. 663–670. ISBN 1-55860-707-2.
65. Ziebart, Brian D.; Maas, Andrew; Bagnell, J. Andrew; Dey, Anind K. (2008-07-13). "Maximum entropy inverse reinforcement learning" (https://dl.acm.org/doi/10.5555/1620270.1620297). *Proceedings of the 23rd National Conference on Artificial Intelligence - Volume 3*. AAAI'08. Chicago, Illinois: AAAI Press: 1433–1438. ISBN 978-1-57735-368-3. S2CID 336219 (https://api.semanticscholar.org/CorpusID:336219).
66. Pitombeira-Neto, Anselmo R.; Santos, Helano P.; Coelho da Silva, Ticiana L.; de Macedo, José Antonio F. (March 2024). "Trajectory modeling via random utility inverse reinforcement learning". *Information Sciences*. **660** 120128. arXiv:2105.12092 (https://arxiv.org/abs/2105.12092). doi:10.1016/j.ins.2024.120128 (https://doi.org/10.1016%2Fj.ins.2024.120128). ISSN 0020-0255 (https://search.worldcat.org/issn/0020-0255). S2CID 235187141 (https://api.semanticscholar.org/CorpusID:235187141).
67. Hayes C, Radulescu R, Bargiacchi E, et al. (2022). "A practical guide to multi-objective reinforcement learning and planning" (https://doi.org/10.1007%2Fs10458-022-09552-y). *Autonomous Agents and Multi-Agent Systems*. **36** 26. arXiv:2103.09568 (https://arxiv.org/abs/2103.09568). doi:10.1007/s10458-022-09552-y (https://doi.org/10.1007%2Fs10458-022-09552-y). S2CID 254235920 (https://api.semanticscholar.org/CorpusID:254235920).,
68. Tzeng, Gwo-Hshiung; Huang, Jih-Jeng (2011). *Multiple Attribute Decision Making: Methods and Applications* (1st ed.). CRC Press. ISBN 978-1-4398-6157-8.
69. Gu, Shangding; Yang, Long; Du, Yali; Chen, Guang; Walter, Florian; Wang, Jun; Knoll, Alois (10 September 2024). "A review of safe reinforcement learning: Methods, theories and applications" (https://kclpure.kcl.ac.uk/portal/files/300373453/A_Review_of_Safe_Reinforcement_Learning_Methods_Theories_and_Applications_2_.pdf) (PDF). *IEEE Transactions on Pattern Analysis and Machine Intelligence*. **46** (12): 11216–11235. Bibcode:2024ITPAM..4611216G (https://ui.adsabs.harvard.edu/abs/2024ITPAM..4611216G). doi:10.1109/TPAMI.2024.3457538 (https://doi.org/10.1109%2FTPAMI.2024.3457538). PMID 39255180 (https://pubmed.ncbi.nlm.nih.gov/39255180).
70. García, Javier; Fernández, Fernando (1 January 2015). "A comprehensive survey on safe reinforcement learning" (https://jmlr.org/papers/volume16/garcia15a/garcia15a.pdf) (PDF). *The Journal of Machine Learning Research*. **16** (1): 1437–1480.
71. Dabney, Will; Ostrovski, Georg; Silver, David; Munos, Remi (2018-07-03). "Implicit Quantile Networks for Distributional Reinforcement Learning" (https://proceedings.mlr.press/v80/dabney18a.html). *Proceedings of the 35th International Conference on Machine Learning*. PMLR: 1096–1105. arXiv:1806.06923 (https://arxiv.org/abs/1806.06923).
72. Chow, Yinlam; Tamar, Aviv; Mannor, Shie; Pavone, Marco (2015). "Risk-Sensitive and Robust Decision-Making: a CVaR Optimization Approach" (https://proceedings.neurips.cc/paper/2015/hash/64223ccf70bbb65a3a4aceac37e21016-Abstract.html). *Advances in Neural Information Processing Systems*. **28**. Curran Associates, Inc. arXiv:1506.02188 (https://arxiv.org/abs/1506.02188).
73. "Train Hard, Fight Easy: Robust Meta Reinforcement Learning" (https://scholar.google.com/citations?view_op=view_citation&hl=en&user=LnwyFkkAAAAJ&citation_for_view=LnwyFkkAAAAJ:eQOLeE2rZwMC). *scholar.google.com*. Retrieved 2024-06-21.

74. Tamar, Aviv; Glassner, Yonatan; Mannor, Shie (2015-02-21). "Optimizing the CVaR via Sampling" (https://ojs.aaai.org/index.php/AAAI/article/view/9561). *Proceedings of the AAAI Conference on Artificial Intelligence*. **29** (1). arXiv:1404.3862 (https://arxiv.org/abs/1404.3862). doi:10.1609/aaai.v29i1.9561 (https://doi.org/10.1609%2Faaai.v29i1.9561). ISSN 2374-3468 (https://search.worldcat.org/issn/2374-3468).
75. Greenberg, Ido; Chow, Yinlam; Ghavamzadeh, Mohammad; Mannor, Shie (2022-12-06). "Efficient Risk-Averse Reinforcement Learning" (https://proceedings.neurips.cc/paper_files/paper/2022/hash/d2511dfb731fa336739782ba825cd98c-Abstract-Conference.html). *Advances in Neural Information Processing Systems*. **35**: 32639–32652. arXiv:2205.05138 (https://arxiv.org/abs/2205.05138).
76. Bozinovski, S. (1982). "A self-learning system using secondary reinforcement". In Trappl, Robert (ed.). Cybernetics and Systems Research: Proceedings of the Sixth European Meeting on Cybernetics and Systems Research. North-Holland. pp. 397–402. ISBN 978-0-444-86488-8
77. Bozinovski S. (1995) "Neuro genetic agents and structural theory of self-reinforcement learning systems". CMPSCI Technical Report 95-107, University of Massachusetts at Amherst [1]
78. Bozinovski, S. (2014) "Modeling mechanisms of cognition-emotion interaction in artificial neural networks, since 1981." Procedia Computer Science p. 255–263
79. Engstrom, Logan; Ilyas, Andrew; Santurkar, Shibani; Tsipras, Dimitris; Janoos, Firdaus; Rudolph, Larry; Madry, Aleksander (2019-09-25). "Implementation Matters in Deep RL: A Case Study on PPO and TRPO" (https://openreview.net/forum?id=r1etN1rtPB). *ICLR*.
80. Colas, Cédric (2019-03-06). "Distributional Soft Actor-Critic with Three Refinements" (https://openreview.net/forum?id=ryx0N3IaIV). *IEEE Transactions on Pattern Analysis and Machine Intelligence*. **47** (5): 3935–3946. arXiv:1904.06979 (https://arxiv.org/abs/1904.06979). Bibcode:2025ITPAM..47.3935D (https://ui.adsabs.harvard.edu/abs/2025ITPAM..47.3935D). doi:10.1109/TPAMI.2025.3537087 (https://doi.org/10.1109%2FTPAMI.2025.3537087). PMID 40031258 (https://pubmed.ncbi.nlm.nih.gov/40031258).
81. Greenberg, Ido; Mannor, Shie (2021-07-01). "Reward Machines: Exploiting Reward Function Structure in Reinforcement Learning" (https://proceedings.mlr.press/v139/greenberg21a.html). *Journal of Artificial Intelligence Research*. **73**. PMLR: 3842–3853. arXiv:2010.11660 (https://arxiv.org/abs/2010.11660). doi:10.1613/jair.1.12440 (https://doi.org/10.1613%2Fjair.1.12440).
82. Uc-Cetina, Víctor; Navarro-Guerrero, Nicolás; Martin-Gonzalez, Anabel; Weber, Cornelius; Wermter, Stefan (2022). "Survey on reinforcement learning for language processing". *Artificial Intelligence Review*. **56**: 1543–1575. arXiv:2104.05565 (https://arxiv.org/abs/2104.05565). doi:10.1007/s10462-022-10205-5 (https://doi.org/10.1007%2Fs10462-022-10205-5).
83. Ranzato, Marc'Aurelio; Chopra, Sumit; Auli, Michael; Zaremba, Wojciech (2015). "Sequence Level Training with Recurrent Neural Networks". arXiv:1511.06732 (https://arxiv.org/abs/1511.06732) [cs.LG (https://arxiv.org/archive/cs.LG)].
84. Paulus, Romain; Xiong, Caiming; Socher, Richard (2017). "A Deep Reinforced Model for Abstractive Summarization". arXiv:1705.04304 (https://arxiv.org/abs/1705.04304) [cs.CL (https://arxiv.org/archive/cs.CL)].
85. Li, Jiwei; Monroe, Will; Ritter, Alan; Jurafsky, Dan; Galley, Michel; Gao, Jianfeng (2016). "Deep Reinforcement Learning for Dialogue Generation". *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. pp. 1192–1202. arXiv:1606.01541 (https://arxiv.org/abs/1606.01541). doi:10.18653/v1/D16-1127 (https://doi.org/10.18653%2Fv1%2FD16-1127).
86. Christiano, Paul F.; Leike, Jan; Brown, Tom B.; Martic, Miljan; Legg, Shane; Amodei, Dario (2017). "Deep Reinforcement Learning from Human Preferences". *Advances in Neural Information Processing Systems*. Vol. 30. arXiv:1706.03741 (https://arxiv.org/abs/1706.03741).

87. Stiennon, Nisan; Ouyang, Long; Wu, Jeffrey; Ziegler, Daniel M.; Lowe, Ryan; Voss, Chelsea; Radford, Alec; Amodei, Dario; Christiano, Paul (2020). "Learning to Summarize from Human Feedback". *Advances in Neural Information Processing Systems*. Vol. 33. pp. 3008–3021. arXiv:2009.01325 (https://arxiv.org/abs/2009.01325).
88. Ouyang, Long; Wu, Jeffrey; Jiang, Xu; et al. (2022). "Training Language Models to Follow Instructions with Human Feedback". *Advances in Neural Information Processing Systems*. Vol. 35. pp. 27730–27744. arXiv:2203.02155 (https://arxiv.org/abs/2203.02155).
89. Rafailov, Rafael; Sharma, Archit; Mitchell, Eric; Ermon, Stefano; Manning, Christopher D.; Finn, Chelsea (2023). "Direct Preference Optimization: Your Language Model is Secretly a Reward Model". *Advances in Neural Information Processing Systems*. Vol. 36. arXiv:2305.18290 (https://arxiv.org/abs/2305.18290).
90. Lambert, Nathan; Morrison, Jacob; Pyatkin, Valentina; et al. (2024). "Tülu 3: Pushing Frontiers in Open Language Model Post-Training". arXiv:2411.15124 (https://arxiv.org/abs/2411.15124) [cs.CL (https://arxiv.org/archive/cs.CL)].
91. OpenAI (2024). "OpenAI o1 System Card". arXiv:2412.16720 (https://arxiv.org/abs/2412.16720) [cs.AI (https://arxiv.org/archive/cs.AI)].
92. DeepSeek-AI; et al. (January 22, 2025). "DeepSeek-R1 incentivizes reasoning in LLMS through reinforcement learning" (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12443585). *Nature*. **645** (8081): 633–638. arXiv:2501.12948 (https://arxiv.org/abs/2501.12948). Bibcode:2025Natur.645..633G (https://ui.adsabs.harvard.edu/abs/2025Natur.645..633G). doi:10.1038/s41586-025-09422-z (https://doi.org/10.1038%2Fs41586-025-09422-z). PMC 12443585 (https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12443585). PMID 40962978 (https://pubmed.ncbi.nlm.nih.gov/40962978).

# Further reading

- Annaswamy, Anuradha M. (3 May 2023). "Adaptive Control and Intersections with Reinforcement Learning" (https://doi.org/10.1146%2Fannurev-control-062922-090153). *Annual Review of Control, Robotics, and Autonomous Systems*. **6** (1): 65–93. doi:10.1146/annurev-control-062922-090153 (https://doi.org/10.1146%2Fannurev-control-062922-090153). ISSN 2573-5144 (https://search.worldcat.org/issn/2573-5144). S2CID 255702873 (https://api.semanticscholar.org/CorpusID:255702873).
- Auer, Peter; Jaksch, Thomas; Ortner, Ronald (2010). "Near-optimal regret bounds for reinforcement learning" (http://jmlr.csail.mit.edu/papers/v11/jaksch10a.html). *Journal of Machine Learning Research*. **11**: 1563–1600.
- Bertsekas, Dimitri P. (2023) [2019]. *Reinforcement Learning and Optimal Control* (http://www.mit.edu/~dimitrib/RLbook.html) (1st ed.). Athena Scientific. ISBN 978-1-886-52939-7.
- Busoniu, Lucian; Babuska, Robert; De Schutter, Bart; Ernst, Damien (2010). *Reinforcement Learning and Dynamic Programming using Function Approximators* (http://www.dcsc.tudelft.nl/rlbook/). Taylor & Francis CRC Press. ISBN 978-1-4398-2108-4.
- François-Lavet, Vincent; Henderson, Peter; Islam, Riashat; Bellemare, Marc G.; Pineau, Joelle (2018). "An Introduction to Deep Reinforcement Learning". *Foundations and Trends in Machine Learning*. **11** (3–4): 219–354. arXiv:1811.12560 (https://arxiv.org/abs/1811.12560). Bibcode:2018arXiv181112560F (https://ui.adsabs.harvard.edu/abs/2018arXiv181112560F). doi:10.1561/2200000071 (https://doi.org/10.1561%2F2200000071). S2CID 54434537 (https://api.semanticscholar.org/CorpusID:54434537).
- Li, Shengbo Eben (2023). *Reinforcement Learning for Sequential Decision and Optimal Control* (https://link.springer.com/book/10.1007/978-981-19-7784-8) (1st ed.). Springer Verlag, Singapore. doi:10.1007/978-981-19-7784-8 (https://doi.org/10.1007%2F978-981-19-7784-8). ISBN 978-9-811-97783-1.

- Powell, Warren (2011). *Approximate dynamic programming: solving the curses of dimensionality* (https://web.archive.org/web/20160731230325/http://castlelab.princeton.edu/adp.htm). Wiley-Interscience. Archived from the original (http://www.castlelab.princeton.edu/adp.htm) on 2016-07-31. Retrieved 2010-09-08.
- Sutton, Richard S. (1988). "Learning to predict by the method of temporal differences" (https://doi.org/10.1007%2FBF00115009). *Machine Learning*. **3** (1): 9–44. Bibcode:1988MLear...3....9S (https://ui.adsabs.harvard.edu/abs/1988MLear...3....9S). doi:10.1007/BF00115009 (https://doi.org/10.1007%2FBF00115009).
- Sutton, Richard S.; Barto, Andrew G. (2018) [1998]. *Reinforcement Learning: An Introduction* (http://incompleteideas.net/sutton/book/the-book.html) (2nd ed.). MIT Press. ISBN 978-0-262-03924-6.
- Szita, Istvan; Szepesvari, Csaba (2010). "Model-based Reinforcement Learning with Nearly Tight Exploration Complexity Bounds" (https://web.archive.org/web/20100714095438/http://www.icml2010.org/papers/546.pdf) (PDF). *ICML 2010*. Omnipress. pp. 1031–1038. Archived from the original (http://www.icml2010.org/papers/546.pdf) (PDF) on 2010-07-14.


## External links

- Dissecting Reinforcement Learning (https://mpatacchiola.github.io/blog/2016/12/09/dissecting-reinforcement-learning.html) Series of blog post on reinforcement learning with Python code
- A (Long) Peek into Reinforcement Learning (https://lilianweng.github.io/posts/2018-02-19-rl-overview/)
- QSMM – reinforcement learning through adaptive probabilistic assembler programs (http://qsmm.org)